

NYC Yellow-Taxi Fleet Allocation — Model & Optimization Documentation

NYC Yellow-Taxi Fleet Allocation — Model & Optimization Documentation

This document describes every modeling and optimization decision implemented in the Python codebase (`engine/`, `run_results.py``). It is derived from reading the source files directly — not from data files.

1. Executive Summary

This project solves a **multi-zone newsvendor fleet allocation** problem:

> Given a fixed fleet of yellow taxis (default **13,000**), how many cabs should be pre-positioned in each of **263 TLC pickup zones** for a given **time bucket** and **day of week**?

The pipeline has two layers:

Layer	What it does	Global optimum?
Demand layer	Estimates latent trip demand per zone using 4 competing approaches	ML layers: no global guarantee
Allocation layer	Solves constrained newsvendor optimization given demand samples	Continuous problem: yes (global) ; integer rounding: heuristic only

There is **no Gurobi, PuLP, or mixed-integer programming**. The inner problem is convex and solved with **SciPy** and optionally **cvxpy (CLARABEL)**.

2. Problem Formulation

2.1 Decision variables

- q_z — number of cabs allocated to zone z , for $z \in \{1, \dots, 263\}$

2.2 Objective (newsvendor cost)

Minimize expected cost of over- and under-stocking:

$$\min_q \sum_z [C_o \cdot E[(q_z - D_z)^+] + C_u \cdot E[(D_z - q_z)^+]]$$

Where:

- $\max(x, 0)$
- D_z = random latent demand in zone z
- Cu_z = **underage cost** (lost revenue per unserved trip)
- Co_z = **overage cost** (idle-cab opportunity cost)

Implementation (`engine/optimize.py`, lines 44–54): sample-average approximation over n historical demand scenarios:

...

$$NV(q) = (1/n) \sum_s \sum_z [Co_z \cdot \max(q_z - D_{\{s,z\}}, 0) + Cu_z \cdot \max(D_{\{s,z\}} - q_z, 0)]$$

...

2.3 Deployment granularity

Optimization is run **once per (time_bucket, day_of_week)** pair — not per week. The fleet of 13,000 is shared across all 263 zones for that slot. Week is used for demand forecasting and backtesting, not for separate allocations per week.

Scenarios evaluated in `run_results.py`: 5 time buckets $\times$ 7 days = **35 deployment slots**.

3. Constraints (Implemented)

All constraints are enforced in `engine/optimize.py` and configured via `engine/config.py` and `engine/bounds.py`.

3.1 Fleet budget (binding inequality)

...

$$\sum_z q_z \leq \text{fleet_size} \text{ (default: 13,000)}$$

...

```
| Solver | Enforcement |
|-----|-----|
| SLSQP | {"type": "ineq", "fun": lambda q: fleet_size - q.sum()} |
| Water-filling | Lagrange multiplier  $\lambda$  bisection on KKT condition |
| cvxpy | `cp.sum(q) <= fleet_size` |
```

3.2 Per-zone floor (minimum allocation)

...

$$q_z \geq \text{floor_z}$$

...

Definition (`engine/bounds.py`, line 53):

...

$$\text{floor_z} = \text{ceil}(\text{floor_alpha} \times \text{median_demand_z})$$

...

Default `floor_alpha = 0.15` (configurable in `OptimizationConfig`).

Demand median is pooled across all training months for the given `(time_bucket, day_of_week)`.

3.3 Per-zone cap (maximum allocation)

'''

$q_z \leq \text{cap}_z$

'''

****Definition**** (`engine/bounds.py`, line 54):

'''

$\text{cap}_z = \text{ceil}(\text{cap_multiplier} \times \text{p95_demand}_z)$

'''

Default `cap_multiplier = 1.5`.

3.4 Box validity

'''

$\text{floor}_z \leq \text{cap}_z$ (enforced after computation and after overrides)

'''

3.5 Feasibility repair (when floors exceed fleet)

If $\sum \text{floor}_z > \text{fleet_size}$, floors are scaled down to at most ****25% of fleet****:

```python

$\text{scale} = (\text{fleet\_size} \times 0.25) / \text{total\_floor}$

$\text{floor}_z \leftarrow \text{floor}(\text{floor}_z \times \text{scale})$

'''

(`engine/bounds.py`, lines 70–75)

### 3.6 Non-negativity

'''

$q_z \geq 0$

'''

Enforced via solver bounds and clipping.

### 3.7 Integer allocation (post-processing, not in optimization)

Continuous solution `q_star` is converted to deployable integers via `_integer_round_repair`:

1. Round each `q_z` to nearest integer
2. Clip to `[floor_z, cap_z]`

3. If  $\sum q_{z} > \text{fleet\_size}$ , greedily subtract from zones furthest above their floor

**This integer step is a heuristic — not guaranteed globally optimal.**

## 3.8 Constraints NOT implemented

The model does **not** include:

- Cab repositioning / travel time between zones
- Multi-period dynamic allocation
- Driver shift constraints
- Integer variables inside the optimizer (no MIP)
- Zone-to-zone coupling beyond the shared fleet budget
- FHV fleet as a decision variable (FHV is in the demand proxy only)

---

## 4. Cost Parameters

Defined in `engine/costs.py`.

| Symbol       | Meaning           | How computed                                                                                                                                                         |
|--------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $Cu_{z,t}$   | Underage cost     | Zone×bucket weighted average of <code>yellow_avg_total_fare</code> (fare clipped at \$200). Missing zones imputed from borough×bucket mean, then global bucket mean. |
| $Co_t$       | Overage cost      | All-zones average fare per time bucket (same for all zones within a bucket)                                                                                          |
| $\tau_{z,t}$ | Critical fractile | $Cu / (Cu + Co)$ , clipped to $[1e-6, 1-1e-6]$                                                                                                                       |

**Economic interpretation:** High-fare zones (e.g., airports) have higher  $\tau$  → allocated more cabs under the newsvendor framework.

**Tunable multipliers** (`OptimizationConfig`): `cu_multiplier`, `co_multiplier`.

---

## 5. Demand Proxy (Input to All Models)

Before any ML model runs, observed pickups are converted to **latent demand** (`engine/proxy.py`):

---

```
mul_factor_b = avg_yellow_b / (avg_yellow_b + avg_fhv_b)
demand_{z,t,d,m} = mul_factor_{b(z)} × (yellow_pickups + fhv_pickups)
```

Where  $b(z)$  is the borough (default), cluster, or global aggregation level.

**Rationale:** Yellow taxis compete with FHV (Uber/Lyft). A zone where yellow is under-represented relative to total market activity signals latent unmet demand.

Default aggregation: **borough-level** (`proxy_aggregation = "borough"`).

---

## 6. Models Used

The project compares **four demand/allocation tracks**. All tracks feed into the **same inner newsvendor solver**.

### 6.1 Track 0 — Empirical Baseline (no ML)

**File:** `engine/models/empirical.py`

| Item                   | Detail                                                                              |
|------------------------|-------------------------------------------------------------------------------------|
| Algorithm              | Historical demand quantiles / bootstrap samples                                     |
| Training               | Pools all training months per <code>(PULocationID, time_bucket, day_of_week)</code> |
| Samples                | Up to 50 bootstrap-aligned scenarios per zone                                       |
| Unconstrained solution | <code>q*_z = quantile(demand_z, <math>\tau_z</math>)</code>                         |
| Role                   | Zero-ML reference; all ML tracks should beat this on realized NV cost               |

### 6.2 Track A1 — Poisson GLM (Predict-then-Optimize)

**File:** `engine/models/linear_demand.py`

| Item                  | Detail                                                                                                                        |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Model                 | <code>sklearn.linear_model.PoissonRegressor</code> (L2, $\alpha=1e-4$ )                                                       |
| Features              | One-hot: <code>PULocationID</code> , <code>time_bucket</code> , <code>day_of_week</code> , <code>week</code> ; scaled weather |
| Target                | <code>demand</code> (from proxy)                                                                                              |
| Paradigm              | <b>PTO</b> — predict $\lambda$ , then <code>q*_z = poisson.ppf(<math>\tau_z</math>, <math>\mu=\lambda_z</math>)</code>        |
| Post-prediction blend | <code>qr_samples = predicted_mean + empirical_deviation</code> before inner solve                                             |

**Optimality:** Poisson GLM loss is convex → sklearn coordinate descent finds the global minimum of the GLM objective. The subsequent PTO step is not jointly optimized with allocation.

### 6.3 Track A2 — LightGBM Quantile Regression (PTO)

**File:** `engine/models/qr_lgbm.py`

| Item           | Detail                                                                                                                             |
|----------------|------------------------------------------------------------------------------------------------------------------------------------|
| Model          | LightGBM with <b>custom per-row pinball objective</b>                                                                              |
| $\tau$ per row | Joined from <code>costs_df</code> on <code>(PULocationID, time_bucket)</code> — 1,315 unique $\tau$ values (263 zones × 5 buckets) |
| Features       | Zone, borough, cluster, cyclical time, weather, encoded time bucket                                                                |
| Defaults       | 300 trees, 63 leaves, <code>lr=0.05</code>                                                                                         |
| Paradigm       | <b>PTO</b> — predict demand quantile, blend with empirical deviation, then inner solve                                             |

**Optimality:** Gradient boosting is **greedy and non-convex**. No global optimum guarantee. Each tree split is locally optimal; the ensemble is not guaranteed globally optimal.

**Note:** `engine/pipeline.py` has LGBM import **commented out** (line 31). Full 4-model bake-off runs only in `run_results.py`.

## 6.4 Track B — Decision-Focused Learning (DFL / SPO+)

**File:** `engine/models/dfl_spo.py`

| Item            | Detail                                                           |
|-----------------|------------------------------------------------------------------|
| Predictor       | Linear: $f_{\theta}(x) = X\theta + b$ per zone                   |
| Outer optimizer | <b>L-BFGS-B</b> via <code>scipy.optimize.minimize</code>         |
| Training epochs | 10 in pipeline; <b>50</b> in <code>run_results.py</code>         |
| Features        | Zone, borough, cluster, cyclical encodings, weather              |
| Training data   | Weekly batches: <code>(features_week, demand_matrix_week)</code> |

**Loss implemented** (lines 123–128):

```
...
NV(D_hat, D_true) = Σ_z [Cu_z · (D_true – D_hat)■ + Co_z · (D_hat – D_true)■]
...
```

Plus L2 regularization on  $\theta$ .

**Important caveat:** The file header cites **SPO+** (Elmachtoub & Grigas 2022), but the implemented loss is a **direct newsvendor cost** with a hand-crafted gradient — not the canonical SPO+ surrogate  $2c \cdot (c^T q(c) - 2c^T q^*(2c - D))$ . The inner solver (`solve_slsqp`, `solve_water_filling`) is **imported but never called during training** (dead code).

**Budget coupling** (lines 135–138): if predicted total demand exceeds fleet, a heuristic penalty shifts gradients.

**Optimality:** L-BFGS-B on a non-convex objective → **local optimum only**. No global guarantee.

---

## 7. Inner Optimization Solvers (Allocation Layer)

**File:** `engine/optimize.py`

All three solvers solve the **same convex problem** (continuous  $q$ ). Default primary solver: **SLSQP**.

### 7.1 Solver 1 — SLSQP (primary)

| Item       | Detail                                                                    |
|------------|---------------------------------------------------------------------------|
| Library    | <code>scipy.optimize.minimize</code> , <code>method="SLSQP"</code>        |
| Gradient   | Analytical: $dNV/dq_z = (Cu_z + Co_z) \cdot F_z(q_z) - Cu_z$              |
| Warm start | Unconstrained $\tau$ -quantile per zone, projected onto budget and bounds |
| Tolerance  | <code>ftol=1e-7</code> , <code>maxiter=1000</code>                        |
| Status     | "optimal" if <code>res.success</code> , else warning                      |

## 7.2 Solver 2 — Water-filling (analytic)

| Item                     | Detail                                                                                      |
|--------------------------|---------------------------------------------------------------------------------------------|
| Method                   | Bisection on Lagrange multiplier $\lambda$ ( <code>scipy.optimize.brentq</code> )           |
| KKT                      | $F_z(q_z) = (Cu_z - \lambda) / (Cu_z + Co_z)$ , clipped to $[\text{floor}_z, \text{cap}_z]$ |
| Docstring claim          | <code>***fastest, exact***</code> — always returns <code>status="optimal"</code>            |
| When budget doesn't bind | Uses unconstrained $\tau$ -quantile solution                                                |

## 7.3 Solver 3 — cvxpy (cross-check)

| Item      | Detail                                                                            |
|-----------|-----------------------------------------------------------------------------------|
| Library   | <code>cvxpy</code> with <code>cp.CLARABEL</code> solver                           |
| Objective | Same newsvendor cost, but uses <code>at most 30 demand samples</code> (speed cap) |
| Purpose   | Independent convex verification                                                   |

## 7.4 Solver cross-check

`compare_solvers()` runs all three and warns if `q*` vectors differ by more than `tol=5.0` cabs per zone. This is the project's `practical global-optimality verification` for the continuous problem.

`run_results.py` treats agreement within 300 cabs as PASS.

---

# 8. Global vs Local Optimum — Detailed Answer

This is the most important section for your review.

## 8.1 Continuous allocation problem → GLOBAL optimum (yes)

The newsvendor cost is `convex` in `q` (explicitly stated in `engine/optimize.py`, line 15). With:

- Convex objective
- Linear inequality constraint (fleet budget)
- Box constraints (floors/caps)

→ Any `local minimum` is a `global minimum` for the continuous relaxation.

| Component        | Global optimum?  | Evidence                                                            |
|------------------|------------------|---------------------------------------------------------------------|
| Water-filling    | <code>Yes</code> | (claimed exact)   KKT bisection on convex problem                   |
| cvxpy + CLARABEL | <code>Yes</code> | (if solver succeeds)   Convex programming theory                    |
| SLSQP            | <code>Yes</code> | (if converges)   Convex problem; warm-started from $\tau$ -quantile |

`Caveat:` SLSQP can fail to converge (`res.success = False`). Water-filling has a fallback if bisection bracket fails.

`Caveat:` cvxpy uses only 30 samples → its objective differs slightly from SLSQP/water-filling → solutions may disagree even though each is globally optimal for `its own` approximate objective.

## 8.2 Integer allocation → NOT global

Rounding + greedy budget repair is a **heuristic**. No branch-and-bound, no MIP, no optimality certificate.

## 8.3 ML demand models → NOT global

| Model          | Optimality                                                             |
|----------------|------------------------------------------------------------------------|
| Poisson GLM    | Global for GLM loss (convex); PTO decouples prediction from allocation |
| LightGBM       | Greedy boosting — local splits, no global guarantee                    |
| DFL (L-BFGS-B) | Non-convex in $\theta$ — <b>local minimum only</b>                     |
| Empirical      | Deterministic given data — no optimization                             |

## 8.4 End-to-end pipeline → NOT globally optimal

Even if the inner solver finds the global allocation for a given demand distribution, the **demand distribution itself is estimated** (with error) and the ML training does not jointly optimize prediction + allocation (except DFL, which uses an approximate surrogate, not the true inner solver).

**Bottom line:** The project guarantees global optimality for the **inner convex allocation step** given fixed demand samples. It does **not** guarantee global optimality for the full predict-then-optimize pipeline or for integer deployments.

---

# 9. End-to-End Pipeline

## 9.1 Data flow

...

TLC raw trips

- eda\_cleaning.py (harmonize yellow + FHV schema)
- zone\_hour\_aggregate.py (zone × hour counts + weather)
- build\_zone\_time\_bucket\_dow\_month\_agg.py
- updated\_2025\_agg.parquet

updated\_2025\_agg.parquet

- engine/data.py (borough map, KMeans clusters, cyclical features)
- engine/proxy.py (latent demand)
- temporal\_split (train wk 1–44, val 45–48, test 49–52)
- engine/costs.py ( $C_u$ ,  $C_o$ ,  $\tau$ )
- engine/bounds.py (floors, caps)
- [demand model] → demand sample matrix
- engine/optimize.py →  $q_{\text{star}}$  (263-vector)
- engine/evaluate.py (backtest metrics)

...

## 9.2 Entry points

| Script                          | Purpose                                                                         |
|---------------------------------|---------------------------------------------------------------------------------|
| <code>engine/pipeline.py</code> | → <code>run_pipeline()</code>   Single-scenario pipeline: empirical + GLM + DFL |
| <code>run_results.py</code>     | Full bake-off: all 4 models × 35 scenarios, 15 charts, CSV outputs              |
| <code>engine/config.py</code>   | Pydantic config for scenario tuning (fleet, bounds, costs, solver)              |

## 9.3 PTO pattern (GLM, LGBM)

1. Train demand model on training weeks
2. Predict mean/quantile demand per zone for scenario
3. Build sample matrix: `predicted_mean + (empirical_samples - empirical_mean)`
4. Pass samples to `optimize.solve()`
5. Deploy `q_star_int`

## 9.4 DFL pattern

1. Train linear  $\theta$  via L-BFGS-B minimizing regret-style loss on weekly batches
2. Predict `D_hat` per zone
3. Same empirical-deviation blend as PTO
4. Pass to same inner `optimize.solve()`

---

# 10. Evaluation Metrics

**File:** `engine/evaluate.py`

| Metric                   | Formula / meaning                                                                  |
|--------------------------|------------------------------------------------------------------------------------|
| <b>NV cost</b> (primary) | $\sum_z [Cu_z \cdot (D_z - q_z) + Co_z \cdot (q_z - D_z)]$ on realized test demand |
| Pinball loss             | Quantile loss at $\tau$                                                            |
| Fill rate                | $\sum \min(q, D) / \sum D$                                                         |
| SPO regret               | $NV(q, D) - NV(q_{oracle}, D)$ where $q_{oracle} = D$ (perfect foresight per zone) |
| Fleet utilization        | $\sum q / \text{fleet\_size}$                                                      |

**Bake-off:** Models ranked by mean NV cost across all test scenarios (`bakeoff_summary`).

**Baselines in `run_results.py`:**

- Optimized allocation (empirical/GLM/LGBM/DFL)
- Status quo (proportional to historical yellow pickups)
- Uniform (fleet / 263 zones)

---

# 11. Configuration Defaults

From `engine/config.py`:

| Parameter | Default | Meaning |
|-----------|---------|---------|
|           |         |         |

```
`fleet_size`	13,000	Total cabs
`floor_alpha`	0.15	Floor = 15% of median demand
`cap_multiplier`	1.5	Cap = 150% of p95 demand
`proxy_aggregation`	`"borough"`	Demand proxy granularity
`n_clusters`	5	KMeans clusters (if using cluster proxy)
`solver`	`"slsqp"`	Primary inner solver
`time_bucket`	`"09:00-15:59"`	Default scenario
`day_of_week`	1 (Monday)	Default scenario
`week`	26	Default forecast week
```

---

## 12. Known Gaps & Review Items

These are important for a thorough model audit:

1. **DFL  $\neq$  canonical SPO** — Loss and gradient are approximations; inner solver not used in training loop.
2. **Integer allocation is heuristic** — No MIP; deployed ``q_star_int`` may differ meaningfully from ``q_star``.
3. **cvxpy subsamples to 30 scenarios** — Cross-check may show false disagreement.
4. **``pipeline.py`` vs ``run_results.py``** — Pipeline omits LGBM; full results script is the authoritative bake-off.
5. **PTO hybrid blending** — All ML tracks add empirical deviation to point predictions; not fully consistent with parametric demand models.
6. **Weather what-if incomplete** — ``weather_override`` config exists but rainy-day scenario in ``run_results.py`` does not re-fit or re-predict with overridden weather.
7. **SPO regret oracle** — Uses ``q_oracle = D`` (perfect per-zone foresight), not the constrained optimal allocation — optimistic benchmark.
8. **No repositioning** — Static pre-positioning only; cabs don't move between zones during the slot.
9.  **$\tau$  in LGBM** — Varies by zone $\times$ bucket but not by DOW (by design in ``qr_lgbm.py`` comments).
10. **Costs from training data only** — Cu/Co computed on weeks 1–44; not updated for test period fare shifts.

---

## 13. Python File Reference

```
File	Role
`engine/config.py`	All tunable parameters (Pydantic)
`engine/data.py`	Load parquet, borough/cluster features, train/val/test split
`engine/proxy.py`	Latent demand from yellow + FHV pickups
`engine/costs.py`	Cu, Co, τ computation
`engine/bounds.py`	Per-zone floors and caps
`engine/optimize.py`	Core newsvendor solver (SLSQP, water-filling, cvxpy)
`engine/evaluate.py`	Metrics and bake-off tables
`engine/pipeline.py`	End-to-end orchestration
`engine/models/empirical.py`	Zero-ML baseline
`engine/models/linear_demand.py`	Poisson GLM (PTO)
`engine/models/qr_lgbm.py`	LightGBM quantile (PTO)
`engine/models/dfl_spo.py`	Linear DFL (L-BFGS-B)
`run_results.py`	Full experiment: 4 models, 35 scenarios, charts
```

```
`eda_cleaning.py`	Upstream trip cleaning (not in optimization loop)
`eda_trips.py`	EDA script
`build_zone_time_bucket_dow_month_agg.py`	Aggregation to engine input format
`zone_hour_aggregate.py`	Hour-level aggregation
```

---

## 14. Quick Reference — Key Formulas

---

```
Demand proxy: demand = mul_factor × (yellow_pickups + fhv_pickups)
Critical fractile: $\tau_z = C_u z / (C_u z + C_o z)$
Unconstrained q^* : $q^*_z = F_z^{-1}(\tau_z)$
Floor: floor_z = ceil($0.15 \times \text{median_demand_z}$)
Cap: cap_z = ceil($1.5 \times \text{p95_demand_z}$)
NV cost: $C_o \cdot (q - D)^+ + C_u \cdot (D - q)^+$
Fleet constraint: $\sum q_z \leq 13,000$
```

---

\*Generated from source code review. For questions about a specific function, see the cited file and line numbers in the repository.\*